

Emotion Classification in Texts using Transformer-based BERT Model

This implementation classifies text into five emotion categories (joy, sadness, anger, fear, neutral) using the BERT transformer architecture with transfer learning. The approach leverages pre-trained BERT embeddings fine-tuned for emotion detection, capturing deep contextual relationships and semantic nuances in text.

The pipeline covers text preprocessing compatible with BERT tokenization, model fine-tuning with learning rate scheduling, and comprehensive evaluation using accuracy, F1 scores, and confusion matrices to assess model performance on emotion classification tasks.

```
In [2]: # Importings

import pandas as pd
import numpy as np

# NumPy 2.0
if not hasattr(np, 'Inf'):
    np.Inf = np.inf
if not hasattr(np, 'NaN'):
    np.NaN = np.nan

import os
os.environ['TF_USE_LEGACY_KERAS'] = '1'

import ktrain
from ktrain import text

import warnings
warnings.filterwarnings('ignore')
```

Configuration and data loading

```
In [4]: class_names = ['joy', 'sadness', 'fear', 'anger', 'neutral']
encoding = {
    'joy': 0,
    'sadness': 1,
    'fear': 2,
    'anger': 3,
    'neutral': 4
}

num_labels = len(class_names)

MODEL_NAME = 'bert'
MAX_LEN = 350
```

```
In [ ]: data_train = pd.read_csv('data/data_train.csv', encoding='utf-8')
data_test = pd.read_csv('data/data_test.csv', encoding='utf-8')
```

```

X_train = data_train.Text.tolist()
y_train_int = [encoding[x] for x in data_train.Emotion.tolist()]

X_test = data_test.Text.tolist()
y_test_int = [encoding[x] for x in data_test.Emotion.tolist()]

data = pd.concat([data_train, data_test], ignore_index=True)

print('size of training set: %s' % (len(data_train['Text'])))
print('size of validation set: %s' % (len(data_test['Text'])))
print(data.Emotion.value_counts())

data.head(10)

```

```

size of training set: 7934
size of validation set: 3393
Emotion
joy      2326
sadness  2317
anger    2259
neutral  2254
fear     2171
Name: count, dtype: int64

```

Out[]:

	Emotion	Text
0	neutral	There are tons of other paintings that I thin...
1	sadness	Yet the dog had grown old and less capable , a...
2	fear	When I get into the tube or the train without ...
3	fear	This last may be a source of considerable disq...
4	anger	She disliked the intimacy he showed towards so...
5	sadness	When my family heard that my Mother's cousin w...
6	joy	Finding out I am chosen to collect norms for C...
7	anger	A spokesperson said : ` Glen is furious that t...
8	neutral	Yes .
9	sadness	When I see people with burns I feel sad, actua...

Tokenizers and data preprocessing

```

In [6]: (x_train, y_train), (x_test, y_test), preproc = text.texts_from_array(
        x_train=X_train, y_train=y_train_int,
        x_test=X_test, y_test=y_test_int,
        class_names=class_names,
        preprocess_mode=MODEL_NAME,
        maxlen=MAX_LEN,
        max_features=MAX_LEN*100)

```

```

preprocessing train...
language: en
done.

```

```

Is Multi-Label? False
preprocessing test...
language: en
done.
task: text classification

```

Model Initialization

```

In [7]: model = text.text_classifier(MODEL_NAME, train_data=(x_train, y_train), preproc=

Is Multi-Label? False
maxlen is 350
done.

```

Training parameters

```

In [6]: learner = ktrain.get_learner(model, train_data=(x_train, y_train),
                                     val_data=(x_test, y_test),
                                     batch_size=6)

```

Training and Evaluation

```

In [7]: learner.fit_onecycle(2e-5, 3)

```

begin training using onecycle policy with max lr of 2e-05...

Epoch 1/3

1323/1323 [=====] - 6253s 5s/step - loss: 0.9456 - accuracy: 0.6352 - val_loss: 0.6144 - val_accuracy: 0.7819

Epoch 2/3

1323/1323 [=====] - 5836s 4s/step - loss: 0.4419 - accuracy: 0.8493 - val_loss: 0.5422 - val_accuracy: 0.8126

Epoch 3/3

1323/1323 [=====] - 8836s 7s/step - loss: 0.1869 - accuracy: 0.9420 - val_loss: 0.5592 - val_accuracy: 0.8243

Out[7]: <tf.keras.src.callbacks.History at 0x18ca9774590>

```

In [ ]: learner.validate(val_data=(x_test, y_test),
                        class_names=class_names)

```

107/107 [=====] - 1947s 18s/step

	precision	recall	f1-score	support
joy	0.85	0.85	0.85	707
sadness	0.79	0.83	0.81	676
fear	0.87	0.84	0.85	679
anger	0.81	0.77	0.79	693
neutral	0.80	0.83	0.81	638
accuracy			0.82	3393
macro avg	0.82	0.82	0.82	3393
weighted avg	0.83	0.82	0.82	3393

```
Out[ ]: array([[604, 18, 15, 15, 55],
               [ 16, 561, 28, 50, 21],
               [ 18, 39, 567, 36, 19],
               [ 22, 65, 30, 537, 39],
               [ 48, 25, 9, 28, 528]])
```

```
In [8]: def f1_from_cm(confusion_matrix):
        n_classes = confusion_matrix.shape[0]
        f1_scores = []

        for i in range(n_classes):
            tp = confusion_matrix[i, i]                # True Positive
            fp = np.sum(confusion_matrix[i, :]) - tp    # False Positive
            fn = np.sum(confusion_matrix[:, i]) - tp    # False Negative

            precision = tp / (tp + fp) if (tp + fp) > 0 else 0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0

            f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

            f1_scores.append(f1)

            print(f"Class {i}: Precision={precision:.4f}, Recall={recall:.4f}, F1={f1:.4f}")

        return f1_scores

cm = np.array([[604, 18, 15, 15, 55],
               [ 16, 561, 28, 50, 21],
               [ 18, 39, 567, 36, 19],
               [ 22, 65, 30, 537, 39],
               [ 48, 25, 9, 28, 528]])
f1_scores = f1_from_cm(cm)

macro_f1 = np.mean(f1_scores) * 100
print(f"BERT average F1 score: {macro_f1:.2f}%")
```

```
Class 0: Precision=0.8543, Recall=0.8531, F1=0.8537
Class 1: Precision=0.8299, Recall=0.7924, F1=0.8107
Class 2: Precision=0.8351, Recall=0.8737, F1=0.8539
Class 3: Precision=0.7749, Recall=0.8063, F1=0.7903
Class 4: Precision=0.8276, Recall=0.7976, F1=0.8123
BERT average F1 score: 82.42%
```

```
In [ ]: predictor = ktrain.get_predictor(learner.model, preproc)
        predictor.get_classes()
        predictor.save('models/bert_model')
```

```
In [ ]: import time

        message = 'I just broke up with my boyfriend'

        start_time = time.time()
        prediction = predictor.predict(message)

        print('predicted: {} ({:.2f}s)'.format(prediction, (time.time() - start_time)))
```

```
predicted: sadness (0.26)
```